Cross-Site Scripting (XSS)

Week 6 Notes | OWASP Top 10 | DG Cyber Security Internship | Diksha Uma Shankar Rao

What it is

XSS is a client-side injection flaw. The application includes untrusted input in a web page without proper encoding, so an attacker's JavaScript runs inside another user's browser with the trust of the vulnerable site (its cookies, session and page content).

Types

- **Reflected XSS:** the payload is part of the request (URL or form) and is sent straight back in the response. It runs only for the person who opens the crafted link.
- **Stored XSS:** the payload is saved on the server (for example in a comment or guestbook) and runs for every user who views that page. Usually more dangerous.
- **DOM-based XSS:** the flaw is in client-side JavaScript that writes unsafe data into the page, so the server response itself may be clean.

Example from the DVWA lab

A simple test script was entered in the Reflected XSS name field and in the Stored XSS guestbook:

```
<script>alert('XSS')</script>
```

The alert box appeared, proving the input was not encoded. With Stored XSS the alert appeared again after refreshing the page, showing the script was saved in the database.

Impact

- Stealing session cookies and taking over accounts
- Defacing pages or showing fake login forms (phishing)
- Capturing keystrokes or redirecting users to malicious sites
- Performing actions on behalf of the victim

Risk ratings in this project

- Reflected XSS: Medium (needs the victim to open a crafted link)
- Stored XSS: High (affects every visitor automatically)

Prevention

- **Encode output** for the context it is placed in (HTML, attribute, JavaScript, URL)
- Validate and sanitise input on the server side
- Set a **Content-Security-Policy** header to restrict which scripts can run
- Mark session cookies **HttpOnly**, **Secure** and **SameSite**
- Avoid unsafe DOM methods such as innerHTML; use textContent instead
- Use frameworks that auto-escape output by default

Key takeaway

Treat all user input as untrusted and encode it every time it is written back into a page.